Why Production Is Slow.

Name _____ Date _____

Six queries below are real shapes that ship to production every day. For each one, **estimate how many rows the database has to look at**, then write a version that does less work and say which index it needs. **Estimate before you run anything** — one of the six cannot be fixed by rewriting it at all.

THE DATABASE

users	200,000
id · email · name · country · created_at	
workspaces	5,000
id · name	
projects	40,000
id · workspace_id · name · created_at	
collaborators	280,320
id · project_id · user_id	
tasks	2,000,000
id · project_id · assignee_id · estimate_hours · status · created_at	

INDEXES THAT ALREADY EXIST

every primary key
`tasks(created_at)` `users(email)`
`tasks(project_id)` `collaborators(project_id)`
not indexed: `tasks.assignee_id`, `projects.workspace_id`, `tasks.estimate_hours`

HOW TO ESTIMATE

read every row of a table	that many rows
find by index, equality	about 1, plus the rows that match
join, no index on the key	rows(A) × rows(B)
join, index on the key	rows(A) × matches each
subquery in SELECT or WHERE	it runs **once per outer row**
sort with no index	every row, then sorted

RUN IT AND CHECK YOURSELF

```
git clone https://github.com/mchoi-altitude/\
  training_slow-queries-lab
cd training_slow-queries-lab
docker compose up -d --build
docker compose exec db psql -U lab -d collab
collab=# EXPLAIN <your query>;
```

Needs **Docker Desktop**. First start seeds 2M tasks, about a minute. **EXPLAIN** prints what the planner thinks each step will cost — compare it with the number you wrote down.

1 Count each user's tasks

```
SELECT u.name,
  (SELECT count(*) FROM tasks t
   WHERE t.assignee_id = u.id) AS n
FROM users u;
```

rows examined _____
a better query

now _____ index to add _____

2 Tasks created on one day

```
SELECT count(*) FROM tasks
WHERE DATE(created_at) = '2026-03-15';
```

rows examined _____
a better query

now _____ index to add _____

3 Users on gmail

```
SELECT * FROM users
WHERE email LIKE '%@gmail.com';
```

rows examined _____
a better query

now _____ index to add _____

4 The ten longest tasks

```
SELECT * FROM tasks
ORDER BY estimate_hours DESC
LIMIT 10;
```

rows examined _____
a better query

now _____ index to add _____

5 Users never assigned a task

```
SELECT * FROM users
WHERE id NOT IN
  (SELECT assignee_id FROM tasks);
```

rows examined _____
a better query

now _____ index to add _____

6 Every task in one workspace

```
SELECT count(*)
FROM tasks t, projects p
WHERE p.id = t.project_id
  AND p.workspace_id = 1;
```

rows examined _____
a better query

now _____ index to add _____